

SQL PLACEMENT PREPARATION

Priority 2 & 3 — JOINS and SUBQUERIES

Goal: Understand the concepts first, then learn the common query patterns used in campus-placement SQL questions.

PART A — PRIORITY 2: JOINS

1. What is a JOIN?

A **JOIN** combines rows from two or more related tables using a matching condition. In real databases, related information is often stored in separate tables.

Example tables:

Employee	Department
emp_id name dept_id	dept_id dept_name
101 Anu 10	10 IT
102 Ravi 20	20 HR
103 Kiran 10	30 Sales

The common relationship is **Employee.dept_id = Department.dept_id**.

2. JOIN Syntax

```
SELECT columns FROM table1 t1 JOIN table2 t2 ON t1.key = t2.key;
```

ON tells SQL how the rows in the two tables are related. Aliases such as e and d make queries shorter.

3. INNER JOIN

Meaning: Returns only rows where the JOIN condition matches in both tables.

```
SELECT e.name, d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Result concept: Anu → IT, Ravi → HR, Kiran → IT. A department with no employee and an employee with no matching department are not included.

Memory: INNER JOIN = matching rows only.

4. LEFT JOIN

Meaning: Returns every row from the left table plus matching rows from the right table. If there is no match, right-side columns become NULL.

```
SELECT e.name, d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id;
```

Use it when: You must keep every record from the first/left table.

Memory: LEFT JOIN = all left + matching right.

5. RIGHT JOIN

Meaning: Returns every row from the right table plus matching rows from the left table.

```
SELECT e.name, d.dept_name
FROM Employee e
RIGHT JOIN Department d
```

```
ON e.dept_id = d.dept_id;
```

Think of RIGHT JOIN as the mirror image of LEFT JOIN. Availability and syntax can vary across database systems.

6. FULL OUTER JOIN

Meaning: Returns matching rows plus unmatched rows from both tables. Where supported, unmatched columns on the other side are NULL.

```
SELECT e.name, d.dept_name
FROM Employee e
FULL OUTER JOIN Department d
ON e.dept_id = d.dept_id;
```

Memory: FULL = **everything from both sides**.

7. SELF JOIN

A **SELF JOIN** joins a table to itself. It is useful when rows in the same table have a relationship, such as employee → manager.

```
SELECT e.name AS employee, m.name AS manager
FROM Employee e
JOIN Employee m
ON e.manager_id = m.emp_id;
```

Why aliases? The same table appears twice, so e and m give the two copies different names.

8. JOIN Comparison

JOIN	What it returns	Memory
INNER	Only matching rows	Match
LEFT	All left + matching right	Keep left
RIGHT	All right + matching left	Keep right
FULL OUTER	All rows from both sides	Keep both
SELF	A table joined to itself	Same table twice

9. Most Important Placement Pattern — Find Records With No Match

Example: Find customers who never ordered.

```
SELECT c.*
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

How it works: LEFT JOIN keeps every customer. Customers with no matching order get NULL in the Orders columns. IS NULL keeps only those customers.

■ **Remember this pattern:** LEFT JOIN + right_table.key IS NULL → **find records with no matching record**.

10. JOIN + WHERE

```
SELECT e.name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.salary > 60000;
```

JOIN first combines related information; WHERE then filters the resulting rows.

11. JOIN + GROUP BY

Find employee count department-wise:

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Why LEFT JOIN? It allows departments with zero employees to remain in the result.

12. Placement Traps — JOINS

- Do not confuse **ON** with **WHERE**: ON defines the relationship; WHERE filters rows.
- INNER JOIN can remove unmatched rows.
- LEFT JOIN keeps every left-table row.
- A foreign key is often the column used for joining, but JOINS are based on a logical matching condition—not simply on the words 'primary key' and 'foreign key'.
- SELF JOIN requires aliases so the two roles of the same table can be distinguished.

PART B — PRIORITY 3: SUBQUERIES

1. What is a Subquery?

A **subquery** is a SQL query written inside another SQL query. The inner query produces a value or set of values that the outer query uses.

Basic structure:

```
SELECT ...  
FROM ...  
WHERE column operator (SELECT ...);
```

Think: **inner query gives information** → **outer query uses that information**.

2. Simple Subquery Example

Find employees earning more than the average salary:

```
SELECT name, salary  
FROM Employee  
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Step 1: Inner query calculates AVG(salary).

Step 2: Outer query compares each employee's salary with that average.

3. Single-Value / Scalar Subquery

A subquery that produces one value can be used with operators such as =, >, <, >= and <=.

Find employees earning the maximum salary:

```
SELECT *  
FROM Employee  
WHERE salary = (SELECT MAX(salary) FROM Employee);
```

The inner query returns one maximum value; the outer query finds rows equal to it.

4. Subquery With IN

IN is useful when the inner query returns multiple possible values.

Example: Find employees who work in departments located in a selected set of department IDs:

```
SELECT name  
FROM Employee  
WHERE dept_id IN (SELECT dept_id FROM Department WHERE location = 'Vijayawada');
```

Memory: = is commonly for one value; IN is useful for a set of values.

5. EXISTS

EXISTS checks whether the subquery returns at least one row. It is commonly used to test whether a related record exists.

Find departments that have at least one employee:

```
SELECT d.dept_name  
FROM Department d  
WHERE EXISTS (  
  SELECT 1  
  FROM Employee e  
  WHERE e.dept_id = d.dept_id  
);
```

If the inner query finds a matching employee, EXISTS is true for that department.

6. NOT EXISTS

Used when you want the opposite: there must be no matching row.

Find customers who never ordered:

```
SELECT c.*
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1 FROM Orders o
  WHERE o.customer_id = c.customer_id
);
```

This is another standard solution to the 'never ordered' problem.

7. Correlated Subquery

A **correlated subquery** refers to a column from the outer query. Therefore, the inner query depends on the current outer row.

Example: Find employees earning more than their department's average salary:

```
SELECT e.name, e.salary, e.department
FROM Employee e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM Employee e2
  WHERE e2.department = e.department
);
```

Key idea: The inner AVG is calculated for the current employee's department.

8. Subquery With GROUP BY / HAVING

Find departments whose average salary is greater than 60,000:

```
SELECT department
FROM Employee
GROUP BY department
HAVING AVG(salary) > 60000;
```

This is not necessarily a subquery, but it is an important placement pattern because GROUP BY/HAVING often replaces a subquery.

9. Subquery in FROM

A subquery can act like a temporary result table (often called a derived table).

Example: Rank salaries and then filter the result:

```
SELECT name, salary
FROM (
  SELECT name, salary,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) x
WHERE rnk <= 3;
```

The inner query creates the ranked result; the outer query filters it.

10. Subquery vs JOIN

Subquery	JOIN
Query inside another query.	Combines rows from tables using a join condition.
Useful for scalar comparisons, EXISTS tests, and nested logic.	Useful when you need columns from related tables.
Can sometimes be rewritten as a JOIN.	Can sometimes replace a subquery depending on the problem.

Do not memorize that one is always 'better'. Choose the form that clearly expresses the required logic and fits the database system.

11. Placement Salary Patterns Using Subqueries

Second highest salary:

```
SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Above average:

```
SELECT * FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Maximum salary:

```
SELECT * FROM Employee WHERE salary = (SELECT MAX(salary) FROM Employee);
```

These three patterns are worth practicing until you can write them without looking.

12. JOIN vs EXISTS — A Common Question

If the question asks you to **return columns from both related tables**, a JOIN is often natural.

If the question asks only whether a related row **exists or does not exist**, EXISTS / NOT EXISTS is often a natural solution.

Example wording:

- 'Show employee and department names' → JOIN
- 'Find departments that have at least one employee' → EXISTS
- 'Find customers who never ordered' → NOT EXISTS or LEFT JOIN + IS NULL

13. Placement Traps — Subqueries

- A scalar subquery used with =, >, < etc. should return a compatible single value; otherwise the query can fail.
- Use IN when the subquery can return multiple values.
- EXISTS tests whether at least one matching row exists; it does not return the matching rows as the main result.
- Correlated subqueries reference the outer query.
- Do not confuse a subquery with a JOIN: they solve related but different query patterns.

FINAL ONE-MINUTE REVISION

Topic	Remember
INNER JOIN	Only matching rows
LEFT JOIN	All left rows + matches
RIGHT JOIN	All right rows + matches
FULL OUTER JOIN	All rows from both sides, where supported
SELF JOIN	Same table joined to itself
LEFT JOIN + IS NULL	Find missing related records
Subquery	Query inside another query
Scalar subquery	Inner query gives one value
IN	Compare against a set of values
EXISTS	At least one related row exists
NOT EXISTS	No related row exists

Topic	Remember
Correlated subquery	Inner query depends on outer row

Practice target: Before moving to the next SQL module, write the JOIN and subquery examples yourself from a blank page. Then solve the salary, customer/order, and department questions without looking at the answers.